



HACKATHON TALK · AGENTIC AI DAY

Google ADK, component by component

A tour of the pieces. You bring the project.



Sitam Meur

AI Engineer, Organiser @ ML Kolkata · September 11, 2026



SPEAKER

Sitam Meur

AI Engineer @ Daily Dose of Data Science

Organiser @ ML Kolkata · GSoC Mentor @ Uramaki Lab

Off duty: cheering for Real Madrid and Mohun Bagan.

BEFORE WE START

What you'll walk out with

ADK ships about a dozen components. A normal project uses four or five.

Each slide is one component. What it does, and when to use it.

THE TWELVE PIECES

- 01 Agent
- 02 Tools
- 03 Built-in tools
- 04 Structured output
- 05 Sub-agents
- 06 Workflow agents
- 07 Agent2Agent (A2A)
- 08 Model Context Protocol (MCP)
- 09 Callbacks
- 10 Sessions and state
- 11 Memory
- 12 Multimodal input

COMPONENT 01

Agent

Agent = Model + **Harness**

The model reasons. The harness is everything around it: instructions, tools, memory, the loop that feeds results back in.

```
weather_agent/agent.py

from google.adk.agents import LlmAgent

# import the tool
from .tools import get_weather

# the Agent class, configured
root_agent = LlmAgent(
    name="weather_agent",
    model="gemini-flash-latest",
    description="Answers questions about the current weather in a city.",
    instruction="Use get_weather for any weather question. Keep replies short.",
    tools=[get_weather],
)
```

In a multi-agent setup, the `description` is what the parent agent reads before deciding to hand off to this one. Write it like an API summary, not a code comment.

COMPONENT 02

Tools

Tools are the hands of the agent. Each one is a Python function you pass in that the model can call.

```
weather_agent/tools.py

def get_weather(city: str) -> dict:
    """
    Get today's weather for a city. Pass the city name as a string.

    Returns a dict: status "ok", temperature condition (float) and sky (str).
    """
    data = weather_api.current(city)
    return {"status": "ok", "temp_c": data.temp, "sky": data.summary}

# pass it straight to the agent
agent = LlmAgent(model="gemini-flash-latest", name="weather_agent",
                 tools=[get_weather])
```

The docstring becomes the tool's schema, so the model knows only what you write there. Anything deterministic you can code yourself belongs here, not in a second agent.

COMPONENT 03

Built-in tools

Check what ADK ships before you write your own search or scraping code.

```
search_agent/agent.py

from google.adk.agents import LlmAgent
from google.adk.tools import google_search    # built-in web search

search_agent = LlmAgent(
    model="gemini-flash-latest",
    name="search_agent",
    tools=[google_search],    # drop it in like any other tool
)
```

Also included: code execution, enterprise search, and wrappers for LangChain (LangchainTool), CrewAI (CrewaiTool), and any OpenAPI spec (OpenAPIToolset).

COMPONENT 04

Structured output

When the next step needs fields instead of prose, hand the agent a schema and it fills that shape.

```
report_agent/agent.py

from pydantic import BaseModel
from google.adk.agents import LlmAgent

class WeatherReport(BaseModel):
    """The shape the agent must return."""
    city: str
    temp_c: float
    sky: str

report_agent = LlmAgent(
    model="gemini-flash-latest", name="report_agent",
    instruction="Fill every field from the conversation so far.",
    output_schema=WeatherReport,    # reply must match this model
    output_key="weather_report",    # saved to session state
)
```

One trade-off. An agent with `output_schema` can't reliably call tools in the same run, except on models like Gemini 3.0. Give formatting its own agent.

COMPONENT 05

Sub-agents

Break one job into a few narrow specialist agents that live in the same project and hand off work between themselves.

```
helpdesk/agent.py

billing = LlmAgent(
    model="gemini-flash-latest", name="billing",
    description="Handles invoices, refunds, and payment status.",
)

support = LlmAgent(
    model="gemini-flash-latest", name="support",
    description="Answers product and account questions.",
)

router = LlmAgent(
    model="gemini-flash-latest", name="router",
    instruction="Send billing questions to billing, the rest to support.",
    sub_agents=[billing, support],    # parent hands off by name
)
```

The parent picks a child by its `description` and hands over control. This fits when one deploy covers every role.

COMPONENT 06

Workflow agents

When you already know the sequence of steps, don't leave it to the model. A workflow agent has no model of its own; it runs its sub-agents in a fixed order.

```
pipeline/agent.py

from google.adk.agents import SequentialAgent, ParallelAgent, LoopAgent

# in list order; each step can read what the last one wrote to state
pipeline = SequentialAgent(name="pipeline", sub_agents=[fetch, summarise, translate])

# all at once; don't let two branches write the same state key
gather = ParallelAgent(name="gather", sub_agents=[news, weather, prices])

# repeats until a child signals done, or max_iterations is hit
refine = LoopAgent(name="refine", sub_agents=[draft, critique], max_iterations=3)
```

SequentialAgent for a pipeline, ParallelAgent for fan-out, LoopAgent for repeat-until-good. They nest, so bigger shapes come from these three.

COMPONENT 07

Agent2Agent (A2A)

Call an agent running in another process, often another team's. It publishes a card of its skills; you connect to that card and use it like a sub-agent.

```
two processes, two files

# their process: wrap their agent and serve it over A2A
from google.adk.a2a.utils.agent_to_a2a import to_a2a

# pricing_agent: an LlmAgent defined elsewhere in their project
a2a_app = to_a2a(pricing_agent, port=8001)
# run: uvicorn pricing_service:a2a_app --port 8001
# it serves an agent card at /.well-known/agent-card.json

# your process: treat that agent card as a remote sub-agent
from google.adk.agents import LlmAgent
from google.adk.agents.remote_a2a_agent import RemoteA2aAgent

pricing = RemoteA2aAgent(
    name="pricing",
    description="Handles pricing and quote questions.",
    agent_card="http://localhost:8001/.well-known/agent-card.json",
)
root_agent = LlmAgent(instruction="Route pricing questions to pricing.", sub_agents=[pricing])
```

Reach for A2A only when a real boundary exists: another team, another stack, another security domain. Inside one codebase, sub-agents are simpler.

COMPONENT 08

Model Context Protocol (MCP)

Connect the agent to tools in another process, local or remote. A local server runs with your privileges, not in a sandbox.

```
files_agent/agent.py

from google.adk.agents import LlmAgent
from google.adk.tools.mcp_tool import McpToolset
from google.adk.tools.mcp_tool.mcp_session_manager import StdioConnectionParams, StreamableHTTPConnectionParams
from mcp import StdioServerParameters

# a local server you spawn: filesystem, sqlite, git, ...
files = McpToolset(
    connection_params=StdioConnectionParams(server_params=StdioServerParameters(
        command="npx", args=["-y", "@modelcontextprotocol/server-filesystem", "/data"])),
    tool_filter=["read_file", "list_directory"], # expose only what you need
)

# a remote server, already deployed, reached over HTTP
crm = McpToolset(connection_params=StreamableHTTPConnectionParams(url="https://mcp.acme.com/mcp"))

root_agent = LlmAgent(
    name="files_agent", model="gemini-flash-latest",
    tools=[files, crm], # pass MCP tools like any other
)
```

Stdio for a server you run next to the agent, Streamable HTTP for one that's already deployed. `tool_filter` keeps you from dumping every tool on the model.

COMPONENT 09

Callbacks

Hooks that run before and after the model, each tool, and each agent.

```
guardrails.py

import re
from google.adk.agents import LlmAgent
from google.adk.agents.callback_context import CallbackContext
from google.adk.models import LlmRequest, LlmResponse

# SECRET_PATTERN: your own regex or detector, defined elsewhere
def strip_secrets(callback_context: CallbackContext, llm_request: LlmRequest):
    """Redact secrets from the request before it reaches the model."""
    for content in llm_request.contents:
        for part in content.parts or []:
            if part.text: # text parts only; tool results and inline data pass through
                part.text = re.sub(SECRET_PATTERN, "[redacted]", part.text)
    return None

root_agent = LlmAgent(
    name="agent", model="gemini-flash-latest",
    before_model_callback=strip_secrets,
)
```

One place to enforce a rule, instead of trusting every prompt to follow it. Guardrails in ADK are callbacks, not a separate component.

COMPONENT 10

Sessions and state

Where this conversation keeps its data. Prefix a key with `user:` or `app:` to make it outlive this session; plain keys stay scoped to the current conversation.

```
app.py

from google.adk.runners import Runner
from google.adk.sessions import InMemorySessionService
from google.adk.tools import ToolContext

# root_agent: imported from agent.py
runner = Runner(agent=root_agent, app_name="trip",
                session_service=InMemorySessionService())

# a tool writes to state; the next turn can read it
def set_city(city: str, tool_context: ToolContext) -> dict:
    """Save the city to session state for later turns to read."""
    tool_context.state["city"] = city    # plain key: this session only
    return {"status": "ok"}
```

A session is one conversation. `state` is a dict that survives every turn in it. Swap `InMemory` for `Database` or `VertexAi` in production.

COMPONENT 11

Memory

Recall that outlasts the session. Give the agent `load_memory` and it can search past conversations, once you have saved them.

```
app.py

from google.adk.agents import LlmAgent
from google.adk.memory import InMemoryMemoryService
from google.adk.runners import Runner
from google.adk.tools import load_memory

# give the agent a way to search past chats
root_agent = LlmAgent(name="agent", model="gemini-flash-latest",
                      tools=[load_memory])

memory_service = InMemoryMemoryService()
runner = Runner(agent=root_agent, app_name="trip",
                session_service=session_service, memory_service=memory_service)

# ADK does not save to memory on its own; call this when a chat ends
# finished_session: a Session with its events populated
await memory_service.add_session_to_memory(finished_session)
```

Swap `InMemoryMemoryService` for `VertexAiMemoryBankService` in production. You also trade keyword matching for semantic recall.

COMPONENT 12

Multimodal input

Send images, audio, and PDFs in the same message as the text. Inline bytes are fine for a photo; past 50 MB for PDFs or 100 MB total, switch to the Files API.

```
intake.py

from google.genai import types

message = types.Content(role="user", parts=[
    types.Part(text="What is damaged in this photo?"),
    types.Part.from_bytes(data=photo_bytes, mime_type="image/jpeg"),
    # same shape for audio ("audio/mp3") and PDFs ("application/pdf")
])

async for event in runner.run_async(user_id="u1", session_id="s1",
                                    new_message=message):
    ...
```

Text and bytes are both parts of one message. A Gemini model reads them together, in the same turn.

THE DEV LOOP

Running it, testing it, shipping it

You write the agent once. The same object runs from four commands: a terminal chat, a local UI, an eval, and a deploy.

```
terminal
```

```
pip install google-adk
```

```
adk create my_agent      # scaffold agent.py, .env, __init__.py
```

```
adk run my_agent         # chat in the terminal
```

```
adk web                  # local UI with a trace of every call
```

```
adk eval my_agent evals/ # score it against saved cases
```

```
adk deploy cloud_run my_agent # or agent_engine, or gke
```

The same agent object runs in the terminal, in the web UI, in an eval, and in production. You change how it launches, not the agent.

PUTTING IT TOGETHER

Which piece for which job

YOU NEED

Deterministic logic you can write yourself

Something ADK already ships, like search or code exec

A strict JSON reply

Several roles in one codebase

A known sequence of steps

An agent another team owns and runs

A tool behind someone else's server

A rule that must run on every call

Data to keep across turns, then across chats

REACH FOR

`a function tool`

`a built-in tool`

`output_schema`

`sub-agents`

`a workflow agent`

A2A

MCP

`a callback`

`session state, then a MemoryService`

Start with one agent and a few function tools. Add a piece when the project asks for it, not before.

LET'S CONNECT

Keep building, keep in touch

NOTEBOOK & CODE

github.com/google/adk-samples

Clone a sample agent, drop your `GOOGLE_API_KEY` into a `.env` file, and run `adk web` to try it.

CONNECT WITH ME

linktr.ee/sitammeur

LinkedIn linkedin.com/in/sitammeur

X / Twitter x.com/sitammeur

FURTHER READING

Where to go from here

ADK documentation

adk.dev

Sample agents to clone and run

github.com/google/adk-samples

adk-python, the SDK source

github.com/google/adk-python

Agent2Agent (A2A) protocol

a2a-protocol.org

Model Context Protocol

modelcontextprotocol.io

Good next step: clone a sample agent, run it with `adk web`, then swap in a tool or a sub-agent of your own.

Thank You



Sitam Meur

AI Engineer @ Daily Dose of Data Science
Organiser @ ML Kolkata · linktr.ee/sitammeur